The Living Archive

Cyfrowe Archiwum Społecznościowe

Dokumentacja projektowa

Kamil Naglik

Spis treści

1 Wprowadzenie **3**
1.1 Opis projektu . . . 3
1.2 Stos technologiczny . . . 3
1.3 Wymagania biznesowe . . . 3
1.4 Wymagania funkcjonalne . . . 3
1.5 Link do repozytorium . . . 4

2 Struktura informacji **4**
2.1 Hierarchia treści i uzasadnienie . . . 4
2.1.1 Jakie problemy rozwiązuje ta struktura? . . . 4
2.2 Nawigacja i orientacja użytkownika . . . 4

3 Architektura systemu **6**
3.1 Schemat architektury . . . 6
3.2 Uzasadnienie architektury . . . 6
3.3 Komunikacja frontend-backend . . . 7
3.4 Bezpieczeństwo . . . 8
3.4.1 Uwierzytelnianie . . . 8
3.4.2 Autoryzacja . . . 8
3.4.3 Walidacja danych . . . 8

4 Model danych **9**
4.1 Schemat modelu danych (ER) . . . 9
4.2 Encje i enumy . . . 9
4.2.1 Enumy . . . 10
4.3 Uzasadnienie rozwiązań bazodanowych . . . 10

5 Struktura metadanych **10**

6 Wyszukiwanie i filtrowanie **11**
6.1 Mechanizm wyszukiwania . . . 11
6.2 Prezentacja wyników . . . 11
6.3 Dodatkowe mechanizmy nawigacji . . . 11

7 Autoryzacja i funkcjonalności administratora **12**
7.1 System ról . . . 12
7.2 Panel administracyjny . . . 12

8 Funkcjonalności twórcy **13**
8.1 Przesyłanie zdjęć . . . 13
8.2 Zarządzanie kolekcją . . . 13

9 GUI i walidacja **13**
9.1 System motywów . . . 13
9.2 Responsywność . . . 14
9.3 Walidacja danych . . . 14

10 Dostępność (WCAG 2.1 AA) **14**

10.1 Podsumowanie zgodności 14
10.2 Szczegółowe rozwiązania dostępności 15

11 Podsumowanie **16**

12 Zrzuty ekranu aplikacji **17**

1 Wprowadzenie

1.1 Opis projektu

The Living Archive to społecznościowe archiwum cyfrowe, które umożliwia użytkownikom przesyłanie, katalogowanie i eksplorację historycznych fotografii z terenu Polski i Europy. Aplikacja łączy tradycyjne podejście archiwistyczne (hierarchia lokalizacji, metadane czasowe) z nowoczesnym interfejsem webowym opartym na mapie interaktywnej.

1.2 Stos technologiczny

- **Frontend:** Angular 19 (standalone components, signals), Tailwind CSS 4, Leaflet.js
- **Backend:** .NET 8, ASP.NET Core Web API, Entity Framework Core
- **Baza danych:** PostgreSQL 16
- **Uwierzytelnianie:** OAuth 2.0 (Google) + JWT z refresh tokenami
- **Mapy:** Leaflet.js z kafelkami CARTO (jasne/ciemne)

1.3 Wymagania biznesowe

WB1. Umożliwienie społeczności digitalizacji i udostępniania prywatnych zbiorów fotograficznych.

WB2. Zapewnienie wielopoziomowej kontroli jakości treści (moderacja administracyjna).

WB3. Dostarczenie intuicyjnych narzędzi wyszukiwania i eksploracji opartych na lokalizacji i czasie.

WB4. Zachowanie dostępności cyfrowej (WCAG 2.1 AA) dla jak najszerszego grona odbiorców.

WB5. Zapewnienie bezpieczeństwa danych użytkowników i ich treści.

1.4 Wymagania funkcjonalne

ID	Opis
WF01	Użytkownik może przeglądać zatwierdzone zdjęcia na mapie interaktywnej i w siatce wyników.
WF02	Użytkownik może wyszukiwać zdjęcia po tytule, opisie, lokalizacji i okresie historycznym.
WF03	Użytkownik może filtrować wyniki kaskadowo: Kraj → Miasto → Dzielnica.
WF04	Zalogowany użytkownik (twórca) może przesyłać zdjęcia w formacie JPG, PNG, TIFF, WebP (do 25 MB).
WF05	Twórca opisuje zdjęcie metadanymi: tytuł, opis, okres historyczny, lokalizacja, współrzędne GPS.
WF06	Twórca może edytować i usuwać własne materiały.

ID	Opis
WF07	Administrator może edytować metadane i usuwać zdjęcia. Zdjęcia są publikowane natychmiast po przesłaniu.
WF08	Administrator może zarządzać użytkownikami (blokowanie/odblokowywanie).
WF09	Administrator może zarządzać hierarchią lokalizacji (CRUD).
WF10	System prowadzi dziennik audytu wszystkich akcji administracyjnych.
WF11	Zalogowani użytkownicy mogą komentować zdjęcia.
WF12	Użytkownicy mogą zgłaszać nieodpowiednie zdjęcia i komentarze. Zgłoszenia widoczne w panelu admina.
WF13	Interfejs dostępny w trzech motywach: jasnym, ciemnym i wysokiego kontrastu.

1.5 Link do repozytorium

`https://github.com/Pyruus/the-living-archive`

2 Struktura informacji

2.1 Hierarchia treści i uzasadnienie

Struktura informacji w The Living Archive opiera się na dwóch osiach organizacyjnych:

1. **Oś przestrzenna** - trzypoziomowa hierarchia lokalizacji: *Kraj* → *Miasto* → *Dzielnica*. Głębokość trzech poziomów jest uzasadniona specyfiką archiwów historycznych - pozwala precyzyjnie lokalizować materiał bez nadmiernej złożoności. Każdy poziom posiada współrzędne geograficzne, co umożliwia wizualizację na mapie.

2. **Oś czasowa** - predefiniowane okresy historyczne (1900-1920, 1920-1939, 1939-1945, 1945-1960, 1960-1980, 1980-2000, 2000-dziś).

2.1.1 Jakie problemy rozwiązuje ta struktura?

- **Problem nawigacji** - kaskadowe filtrowanie (kraj → miasto → dzielnica) prowadzi użytkownika od ogółu do szczegółu, eliminując potrzebę znajomości pełnej ścieżki lokalizacji.

- **Problem odkrywania** - mapa interaktywna pozwala użytkownikom wizualnie eksplorować zasoby archiwum bez konieczności formułowania zapytań tekstowych.

2.2 Nawigacja i orientacja użytkownika

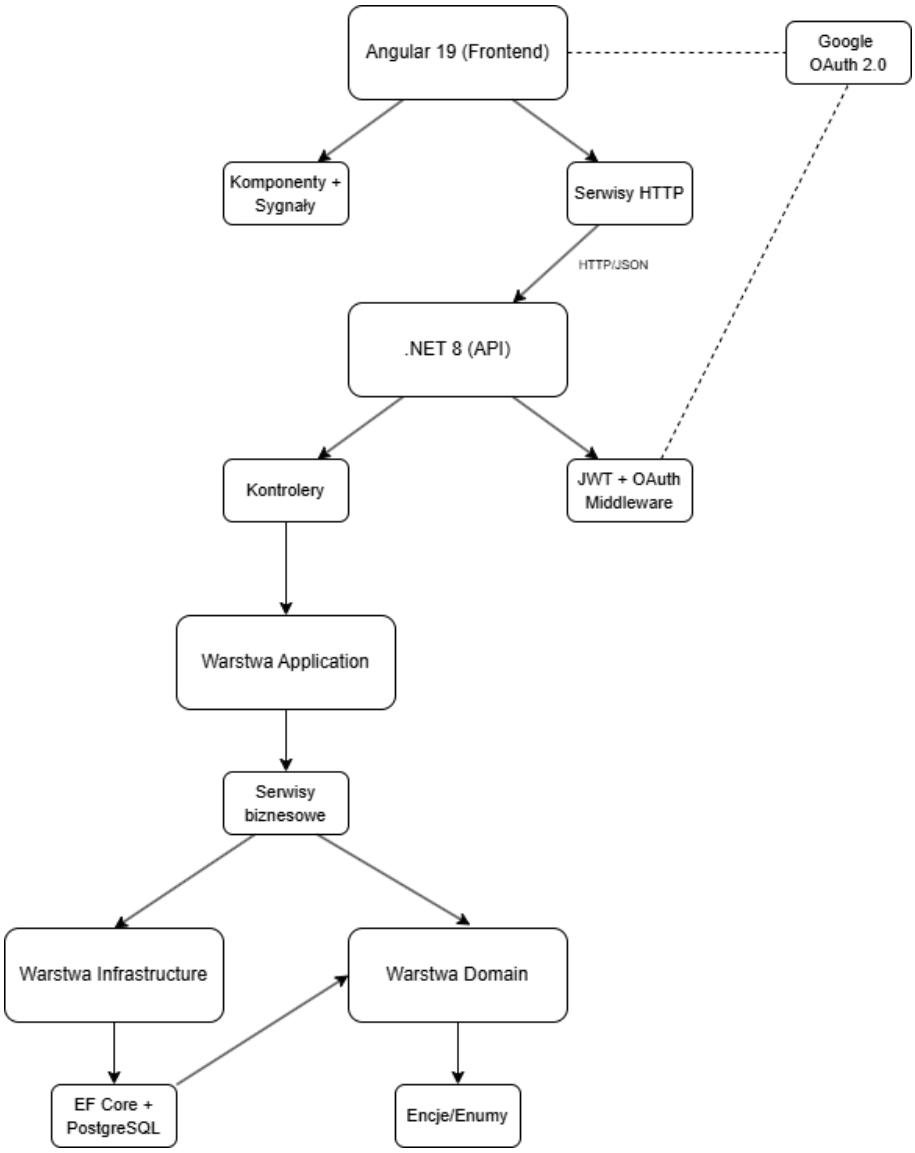
Nawigacja w aplikacji opiera się na kilku wzorcach:

- **Pasek nawigacyjny** - stały, przyklejony do góry strony. Zawiera logo, linki do sekcji (widoczne w zależności od roli) oraz przełącznik motywu. Na urządzeniach mobilnych zwijany do menu hamburgerowego.

- **Nawigacja kaskadowa** - w panelu wyszukiwania trzy powiązane listy rozwijane (kraj → miasto → dzielnica) pojawiają się progresywnie - lista miast jest widoczna dopiero po wyborze kraju.
- **Drzewo lokalizacji** - panel boczny na stronie eksploracji umożliwia przeglądanie hierarchii w formie drzewa z możliwością zwijania/rozwijania węzłów.
- **Mapa interaktywna** - alternatywna forma nawigacji; użytkownik klika na marker, aby zobaczyć zdjęcie lub grupę zdjęć z danej lokalizacji.
- **Kreator wielokrokowy** - przesyłanie zdjęcia podzielone na 4 jasno oznaczone kroki z wizualnym wskaźnikiem postępu.
- **Panel administracyjny z zakładkami** - nawigacja klawiaturowa (strzałki, Home, End) między zakładkami: Zdjęcia, Użytkownicy, Lokalizacje, Dziennik audytu.

3 Architektura systemu

3.1 Schemat architektury



Rysunek 1: Schemat architektury systemu The Living Archive

3.2 Uzasadnienie architektury

System został zaprojektowany zgodnie z zasadami **Clean Architecture**:

- **Warstwa Domain** zawiera encje (`User`, `Photo`, `HierarchyNode`, `AuditLog`, `RefreshToken`) i enumy. Nie ma żadnych zależności zewnętrznych.
- **Warstwa Application** definiuje interfejsy serwisów (`IPhotoService`, `ILocationService`, `IUserService`, `IAuditService`, `IAuditLogQueryService`) oraz DTO. Zależy wyłącznie od Domain.
- **Warstwa Infrastructure** implementuje serwisy, konfiguruje bazę danych (EF Core + PostgreSQL) i obsługuje tokeny JWT. Zależy od Application i Domain.

- **Warstwa API** zawiera kontrolery HTTP, middleware uwierzytelniania i konfigurację DI. Kontrolery delegują logikę do serwisów.

Uzasadnienie: Oddzielenie logiki biznesowej od warstwy HTTP pozwala na łatwe testowanie serwisów, wymianę bazy danych bez zmian w logice, a także wielokrotne użycie serwisów w różnych kontrolerach (np. `ILocationService` używany zarówno przez kontroler publiczny, twórcy jak i administratora).

3.3 Komunikacja frontend-backend

Komunikacja odbywa się wyłącznie przez RESTful API (JSON over HTTP). Frontend korzysta z dedykowanych serwisów HTTP (`PhotoApiService`, `AdminApiService`), które enkapsulują wywołania `HttpClient`. Interceptor `authInterceptor` automatycznie dołącza token JWT do każdego żądania.

Metoda	Endpoint	Opis	Dostęp
Publiczne			
GET	`/api/public/search/photos`	Wyszukiwanie zdjęć	Publiczny
GET	`/api/public/search/photos/{id}`	Szczegóły zdjęcia	Publiczny
GET	`/api/public/search/locations`	Drzewo lokalizacji	Publiczny
GET	`/api/public/comments/{photoId}`	Komentarze zdjęcia	Publiczny
POST	`/api/public/comments`	Dodaj komentarz	Zalogowany
DELETE	`/api/public/comments/{id}`	Usuń komentarz	Zalogowany
POST	`/api/public/comments/report`	Zgłoś komentarz	Zalogowany
POST	`/api/public/comments/report-photo`	Zgłoś zdjęcie	Zalogowany
Uwierzytelnianie			
GET	`/api/auth/login/{provider}`	Logowanie OAuth	Publiczny
GET	`/api/auth/callback`	Callback OAuth	Publiczny
POST	`/api/auth/refresh`	Odświeżenie tokenu	Publiczny
POST	`/api/auth/revoke`	Unieważnienie tokenu	Zalogowany
Twórca			
POST	`/api/creator/photos`	Przesłanie zdjęcia	Twórca
GET	`/api/creator/photos`	Lista własnych zdjęć	Twórca
GET	`/api/creator/photos/{id}`	Szczegóły zdjęcia	Twórca
PUT	`/api/creator/photos/{id}`	Edycja metadanych	Twórca
DELETE	`/api/creator/photos/{id}`	Usunięcie zdjęcia	Twórca
POST	`/api/creator/locations`	Utworzenie lokalizacji	Twórca
Administrator			
GET	`/api/admin/photos`	Lista zdjęć	Admin
PUT	`/api/admin/photos/{id}/metadata`	Edycja metadanych	Admin
PUT	`/api/admin/photos/{id}/status`	Zmiana statusu	Admin
DELETE	`/api/admin/photos/{id}`	Usunięcie zdjęcia	Admin
GET	`/api/admin/users`	Lista użytkowników	Admin
PUT	`/api/admin/users/{id}/block`	Zablokowanie	Admin
PUT	`/api/admin/users/{id}/unblock`	Odblokowanie	Admin
GET	`/api/admin/locations`	Lista lokalizacji	Admin
POST	`/api/admin/locations`	Utworzenie lokalizacji	Admin

Metoda	Endpoint	Opis	Dostęp
PUT	`/api/admin/locations/{id}`	Edycja lokalizacji	Admin
DELETE	`/api/admin/locations/{id}`	Usunięcie lokalizacji	Admin
GET	`/api/admin/reports`	Lista zgłoszeń	Admin
GET	`/api/admin/reports/count`	Liczba nierozwiązanych	Admin
PUT	`/api/admin/reports/{id}/resolve`	Rozwiąż zgłoszenie	Admin
DELETE	`/api/admin/reports/comment/{id}`	Usuń komentarz (soft-delete)	Admin
PUT	`/api/admin/reports/comment/{id}/restore`	Przywróć komentarz	Admin
PUT	`/api/admin/adminphotos/{id}/restore`	Przywróć zdjęcie	Admin
GET	`/api/admin/auditlogs`	Dziennik audytu	Admin

3.4 Bezpieczeństwo

3.4.1 Uwierzytelnianie

System wykorzystuje **OAuth 2.0** z dostawcą Google do logowania. Po uwierzytelnieniu zewnętrznym serwer generuje parę tokenów:

- **Access Token** (JWT, HS256) - ważność 60 minut, zawiera `sub` (ID użytkownika), `email`, `name`, `role`, `isBlocked`.
- **Refresh Token** - ważność 7 dni, przechowywany w bazie danych, możliwy do unieważnienia.

3.4.2 Autoryzacja

Trzy polityki autoryzacji:

Polityka	Wymagania
`RequireCreator`	Zalogowany + rola Creator + niezablokowany
`RequireAdmin`	Zalogowany + rola Admin
`RequireCreatorOrAdmin`	Zalogowany + (Creator lub Admin) + niezablokowany

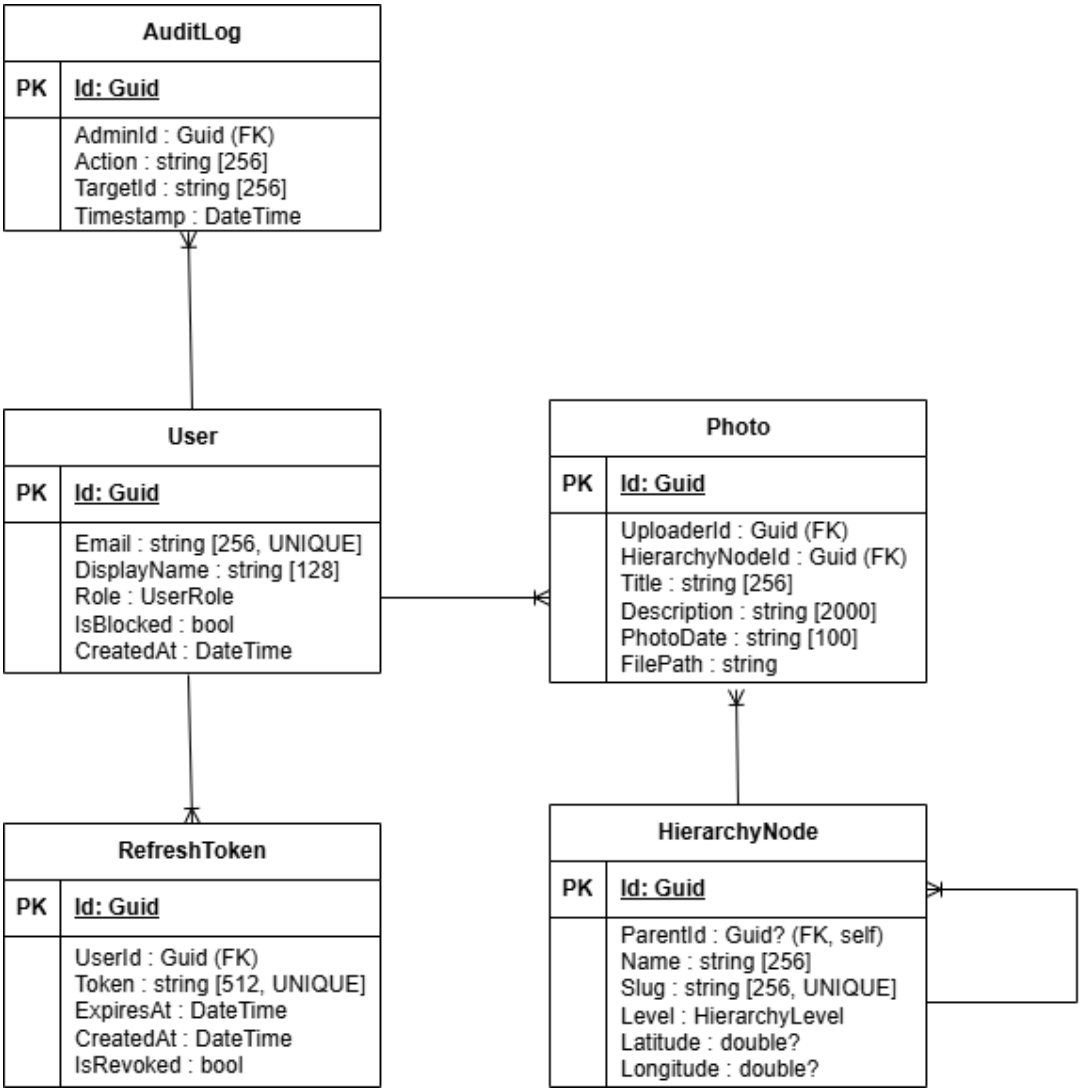
Blokowanie użytkownika jest natychmiastowe - `NotBlockedHandler` sprawdza stan w bazie danych przy każdym żądaniu, niezależnie od zawartości tokenu JWT.

3.4.3 Walidacja danych

- Limit rozmiaru pliku: 25 MB.
- Dozwolone formaty: JPG, JPEG, PNG, TIFF, TIF, WebP.
- Walidacja pól formularza po stronie klienta (Angular) i serwera (serwisy).
- Referencyjna integralność w bazie danych (`RESTRICT` na lokalizacjach, `CASCADE` na użytkownikach).

4 Model danych

4.1 Schemat modelu danych (ER)



Rysunek 2: Diagram ER modelu danych

4.2 Encje i enumy

Oprócz encji przedstawionych na diagramie, system zawiera również:

- **Comment** - komentarze do zdjęć. Pola: `Id`, `PhotoId` (FK), `AuthorId` (FK), `Content` [1000 znaków], `IsReported`, `IsDeleted`, `CreatedAt`. Relacja N:1 z `Photo` i `User`. Usunięcie komentarza jest miękkie (soft-delete) - komentarz pozostaje w bazie danych z flagą `IsDeleted`, co umożliwia jego przywrócenie przez administratora.
- **Report** - zgłoszenia zdjęć i komentarzy. Pola: `Id`, `ReporterId` (FK), `PhotoId?` (FK), `CommentId?` (FK), `Reason` [500 znaków], `IsResolved`, `ResolutionAction?`, `CreatedAt`. Zgłoszenie dotyczy albo zdjęcia, albo komentarza (polimorficzne). Pole `ResolutionAction` przechowuje sposób rozwiązania: `Resolved` (odrzucono zgłoszenie) lub `Deleted` (usunięto zgłoszony element).

4.2.1 Enumy

- **UserRole:** `Viewer`, `Creator`, `Admin`
- **PhotoStatus:** `Pending`, `Approved`, `Rejected`
- **HierarchyLevel:** `Country`, `City`, `District`

4.3 Uzasadnienie rozwiązań bazodanowych

1. **PostgreSQL** - wybrana ze względu na dojrzałe wsparcie dla typów danych (UUID, enum-like, pełnotekstowe), doskonałą integrację z EF Core (Npgsql) oraz bezpłatną licencję.
2. **HierarchyNode z self-referencją** - zamiast trzech osobnych tabel (Country, City, District) zastosowano jedną tabelę z polem `ParentId` i `Level`. Pozwala to na łatwą rozbudowę hierarchii, zunifikowane zapytania i prostszą obsługę relacji rodzic-dziecko.
3. **Współrzędne na lokalizacji, nie na zdjęciu** - współrzędne geograficzne przypisane do `HierarchyNode`, nie do `Photo`. Dzięki temu wiele zdjęć z tej samej lokalizacji dzieli wspólne współrzędne, co ułatwia grupowanie na mapie.
4. **Kaskadowe usuwanie vs. ograniczenia** - usunięcie użytkownika kaskadowo usuwa jego zdjęcia i tokeny (`CASCADE`), ale lokalizacje są chronione (`RESTRICT`) - nie można usunąć lokalizacji z przypisanymi zdjęciami.
5. **Dziennik audytu** - osobna tabela `AuditLog` z indeksem na `AdminId` i `Timestamp` zapewnia szybkie filtrowanie i przejrzystość działań administracyjnych.

5 Struktura metadanych

Każde zdjęcie w archiwum opisane jest następującymi metadanymi:

Pole	Typ	Opis
Title	string (wymagane)	Tytuł zdjęcia, do 256 znaków. Przeszukiwany w wyszukiwarce.
Description	string	Opis kontekstu historycznego, do 2000 znaków. Przeszukiwany w wyszukiwarce.
PhotoDate	string	Okres historyczny (np. `1939-1945`). Służy do filtrowania czasowego.
Location	HierarchyNode (FK)	Węzeł w hierarchii Kraj → Miasto → Dzielnica. Służy do filtrowania przestrzennego.
Współrzędne	(lat, lng)	Dziedziczone z przypisanego `HierarchyNode`. Służą do wyświetlania na mapie.
Status	enum	Stan zdjęcia: Zatwierdzone (domyślny po przesłaniu), Odrzucone (usunięte przez admina).

Pole	Typ	Opis
FilePath	string	Ścieżka do pliku na serwerze (generowana automatycznie).
CreatedAt	DateTime	Data przesłania (automatyczna).
Uploader	User (FK)	Autor przesłania (automatyczny na podstawie sesji).

Przeszukiwane pola: Wyszukiwarka indeksuje `Title` i `Description` (zawieranie tekstu). Filtrowanie odbywa się po `PhotoDate` (okres) i `HierarchyNodeId` (lokalizacja wraz z potomkami - system automatycznie uwzględnia dzieci i wnuki wybranego węzła).

6 Wyszukiwanie i filtrowanie

6.1 Mechanizm wyszukiwania

Wyszukiwarka dostępna na stronie `/explore` oferuje trzy niezależne osie filtrowania, które można łączyć:

1. **Zapytanie tekstowe** - wyszukiwanie w polach `Title` i `Description` metodą `Contains` (case-insensitive). Debouncing 350 ms po stronie frontendu zapobiega nadmiernym zapytaniom.
2. **Okres historyczny** - filtrowanie po polu `PhotoDate`. Predefiniowane przedziały odzwierciedlają kluczowe epoki historyczne.
3. **Lokalizacja** - filtrowanie kaskadowe z trzema poziomami. Po wyborze węzła w hierarchii, system automatycznie uwzględnia wszystkie zdjęcia przypisane do tego węzła *oraz jego potomków* (dzieci i wnuki).

6.2 Prezentacja wyników

Wyniki prezentowane są w dwóch formach:

- **Siatka kart** - zdjęcia wyświetlane w responsywnej siatce (1/2/3 kolumny w zależności od szerokości). Każda karta zawiera miniaturę, lokalizację, datę, tytuł i fragment opisu.
- **Mapa interaktywna** - markery na mapie Leaflet grupowane po współrzędnych. Pojedyncze zdjęcie - ikona aparatu; wiele zdjęć - ikona biblioteki z licznikiem. Kliknięcie otwiera modal ze szczegółami lub listą zdjęć z danej lokalizacji.

Paginacja typu *infinite scroll* - przycisk „Wczytaj więcej wspomnień“ doładowuje kolejne strony wyników.

6.3 Dodatkowe mechanizmy nawigacji

- **Drzewo lokalizacji** (panel boczny) - alternatywna forma filtrowania; kliknięcie na węzeł natychmiast filtruje wyniki.

- **Aktywne filtry** - chipsy nad wynikami pokazują aktywne kryteria z możliwością usunięcia poszczególnych filtrów.
- **Przycisk resetowania** - „Wyczyść filtry“ przywraca stan domyślny.

7 Autoryzacja i funkcjonalności administratora

7.1 System ról

System definiuje trzy role:

Rola	Uprawnienia
Viewer	Przeglądanie zatwierdzonych zdjęć, wyszukiwanie, filtrowanie. Rola domyślna..
Creator	Wszystko co Viewer + przesyłanie zdjęć, edycja i usuwanie własnych materiałów, tworzenie lokalizacji.
Admin	Wszystko co Creator + zarządzanie zdjęciami (edycja metadanych, usuwanie), zarządzanie użytkownikami (blokowanie), zarządzanie lokalizacjami (CRUD), przegląd dziennika audytu.

7.2 Panel administracyjny

Panel administracyjny (`/admin`) podzielony jest na pięć zakładek:

1. **Zgłoszenia** - pierwsza zakładka, wyróżniona kolorem czerwonym z licznikiem nierozwiązanych zgłoszeń. Administrator widzi zgłoszenia zdjęć i komentarzy z bezpośrednimi linkami do zgłoszonych treści. Może usunąć zgłoszony element (miękkie usunięcie) lub oznaczyć zgłoszenie jako rozwiązane. Usunięcie elementu automatycznie rozwiązuje powiązane zgłoszenia. W widoku rozwiązanych zgłoszeń wyświetlany jest status rozwiązania: *Usunięto* (z opcją przywrócenia elementu) lub *Odrzucono zgłoszenie*. Wszystkie akcje administracyjne są rejestrowane w dzienniku audytu.
2. **Zdjęcia** - lista wszystkich zdjęć z filtrami statusu i wyszukiwarką. Administrator może edytować metadane lub usunąć zdjęcie. Każda akcja logowana do dziennika audytu.
3. **Użytkownicy** - lista użytkowników z filtrami (wyszukiwanie, status blokady). Administrator może blokować i odblokowywać konta. Zablokowany użytkownik natychmiast traci dostęp (weryfikacja przy każdym żądaniu).
4. **Lokalizacje** - drzewo lokalizacji w widoku hierarchicznym z możliwością dodawania, edycji i usuwania węzłów. Usuwanie zablokowane, gdy lokalizacja posiada dzieci lub przypisane zdjęcia. Edycja z interaktywną mapą do ustawiania współrzędnych.
5. **Dziennik audytu** - chronologiczna lista wszystkich akcji administracyjnych z informacją o administratorze, typie akcji (z polskimi etykietami), identyfikatorze obiektu i dacie. Rejestrowane akcje: usunięcie/przywrócenie zdjęcia, edycja metadanych, usunięcie/przywrócenie komentarza, rozwiązanie zgłoszenia, blokowanie/odblokowanie użytkownika, zarządzanie lokalizacjami.

8 Funkcjonalności twórcy

8.1 Przesyłanie zdjęć

Kreator przesyłania (`/upload`) prowadzi użytkownika przez cztery kroki:

1. **Plik** - wybór pliku przez kliknięcie lub przeciągnięcie (drag & drop). Walidacja formatu (JPG, PNG, TIFF, WebP) i rozmiaru (maks. 25 MB). Podgląd wybranego pliku.
2. **Metadane** - formularz z polami: tytuł (wymagany, maks. 256 znaków), okres historyczny (lista rozwijana), opis (opcjonalny, maks. 2000 znaków).
3. **Lokalizacja** - przeglądarka hierarchii lokalizacji z możliwością tworzenia nowych węzłów (kraj, miasto, dzielnica) inline. Interaktywna mapa do ustawiania współrzędnych GPS.
4. **Podsumowanie** - przegląd wszystkich danych przed wysłaniem. Po zatwierdzeniu przekierowanie do kolekcji użytkownika.

8.2 Zarządzanie kolekcją

Strona `/collection` prezentuje zdjęcia użytkownika z funkcjami:

- **Statystyki** - liczba zdjęć ogółem, zatwierdzonych i oczekujących.
- **Filtrowanie** - po statusie (wszystkie, zatwierdzone, oczekujące, odrzucone).
- **Sortowanie** - najnowsze, najstarsze, zatwierdzone najpierw.
- **Edycja** - modal edycji metadanych (tytuł, opis, okres).
- **Usuwanie** - z potwierdzeniem; usuwany jest zarówno rekord w bazie, jak i plik z serwera.
- **Wizualne statusy** - kolorowe odznaki: zielona (zatwierdzone), bursztynowa (oczekujące), czerwona (odrzucone). Zdjęcia oczekujące w odcieniach szarości, odrzucone z efektem rozmycia.

9 GUI i walidacja

9.1 System motywów

Aplikacja oferuje trzy motywy kolorystyczne, przełączane przez dedykowany komponent w nagłówku:

Motyw	Paleta	Uwagi
Jasny (domyślny)	Primary: `#003E6F`	Mapy: kafelki CARTO Light
Ciemny	Primary: `#A1C9FF`	Mapy: kafelki CARTO Dark
Wysokiego kontrastu	Primary: `#004A7A`	Mapy: kafelki CARTO Light

Wybór motywu jest persystowany w `localStorage`. Przy pierwszym uruchomieniu respektowane są preferencje systemowe (`prefers-color-scheme`, `prefers-contrast`).

9.2 Responsywność

Interfejs jest w pełni responsywny:

- **Nawigacja** - pasek na desktopie, menu hamburgerowe na urządzeniach mobilnych.
- **Mapa eksploracji** - na desktopie jako tło za panelem wyszukiwania; na mobile jako osobna sekcja poniżej.
- **Siatka wyników** - 1 kolumna (mobile), 2 kolumny (tablet), 3 kolumny (desktop).
- **Panel boczny lokalizacji** - widoczny tylko na ekranach $\geq$ 1024 px, z możliwością zwinięcia.

9.3 Walidacja danych

Walidacja odbywa się na dwóch poziomach:

1. **Frontend (Angular):**
   - Tytuł: wymagany, maks. 256 znaków.
   - Opis: maks. 2000 znaków.
   - Plik: walidacja formatu i rozmiaru przed wysłaniem.
   - Lokalizacja: wymagana.
   - Przyciski „Dalej“ i „Wyślij“ zablokowane do spełnienia warunków.
2. **Backend (serwisy):**
   - Zwracany `ServiceResult<T>` z polem `Error` w przypadku niepowodzenia.
   - Walidacja istnienia lokalizacji przed przypisaniem.
   - Walidacja hierarchii.
   - Walidacja unikalności slugu.
   - Walidacja uprawnień (użytkownik może edytować/usuwać tylko swoje zdjęcia).

Komunikaty o błędach wyświetlane użytkownikowi w języku polskim z atrybutem `role="alert"` dla czytników ekranowych.

10 Dostępność (WCAG 2.1 AA)

10.1 Podsumowanie zgodności

Aplikacja została zaprojektowana z myślą o zgodności ze standardem **WCAG 2.1 na poziomie AA**. Poniżej podsumowanie kluczowych aspektów:

WCAG	Kryterium	Implementacja
1.1.1	Treść nietekstowa	Wszystkie obrazy posiadają atrybut `alt`. Ikony dekoracyjne oznaczone `aria-hidden="true"`.
1.3.1	Informacje i relacje	Semantyczny HTML: `<nav>`, `<main>`, `<footer>`, `<article>`, `<section>`. Formularze z etykietami `<label>`.
1.3.2	Kolejność odczytu	Logiczna kolejność DOM zgodna z wizualną prezentacją.
1.4.1	Użycie koloru	Statusy zdjęć przekazywane kolorem *i* tekstem (np. „Zatwierdzone“).
1.4.3	Kontrast (min.)	Motyw wysokiego kontrastu zapewnia wzmocniony kontrast. Standardowe motywy używają palet Material Design 3.
1.4.11	Kontrast elementów UI	Obramowania, ikony i elementy interaktywne spełniają minimalny kontrast 3:1.
2.1.1	Klawiatura	Cała nawigacja dostępna klawiaturą: Tab, Enter, Space, Escape, strzałki.
2.1.2	Brak pułapki	Modalne okna dialogowe zamykane klawiszem Escape. Focus wraca do elementu wyzwalającego.
2.4.1	Pomijanie bloków	Punkt orientacyjny `<main id="main-content»`.
2.4.2	Tytuł strony	Każda trasa ma opisowy tytuł.
2.4.3	Kolejność fokusu	Logiczna kolejność tabulacji w formularzach i modalach.
2.4.6	Nagłówki i etykiety	Opisowe nagłówki (`h1-h3`) i etykiety formularzy.
3.1.1	Język strony	`<html lang="pl»`.
3.2.1	Przy fokusie	Żadna zmiana kontekstu nie następuje przy samym fokusie.
3.3.1	Identyfikacja błędu	Błędy walidacji wyświetlane z `role="alert"`.
3.3.2	Etykiety / instrukcje	Pola formularzy posiadają etykiety i tekst pomocniczy.
4.1.2	Nazwa, rola, wartość	Elementy interaktywne z odpowiednimi atrybutami ARIA (`aria-label`, `aria-expanded`, `aria-modal`, `aria-selected`, `aria-current`, `role`).

10.2 Szczegółowe rozwiązania dostępności

- **Przełącznik motywu** - zaimplementowany jako `role="radiogroup"` z pełną nawigacją klawiaturową (strzałki, automatyczne przełączanie fokusu). Zmiana motywu ogłaszana czytnikowi ekranowemu przez element `aria-live`.
- **Zakładki administracyjne** - `role="tablist"` / `role="tab"` / `role="tabpanel"`

z klawiaturą: strzałki lewo/prawo, Home, End.

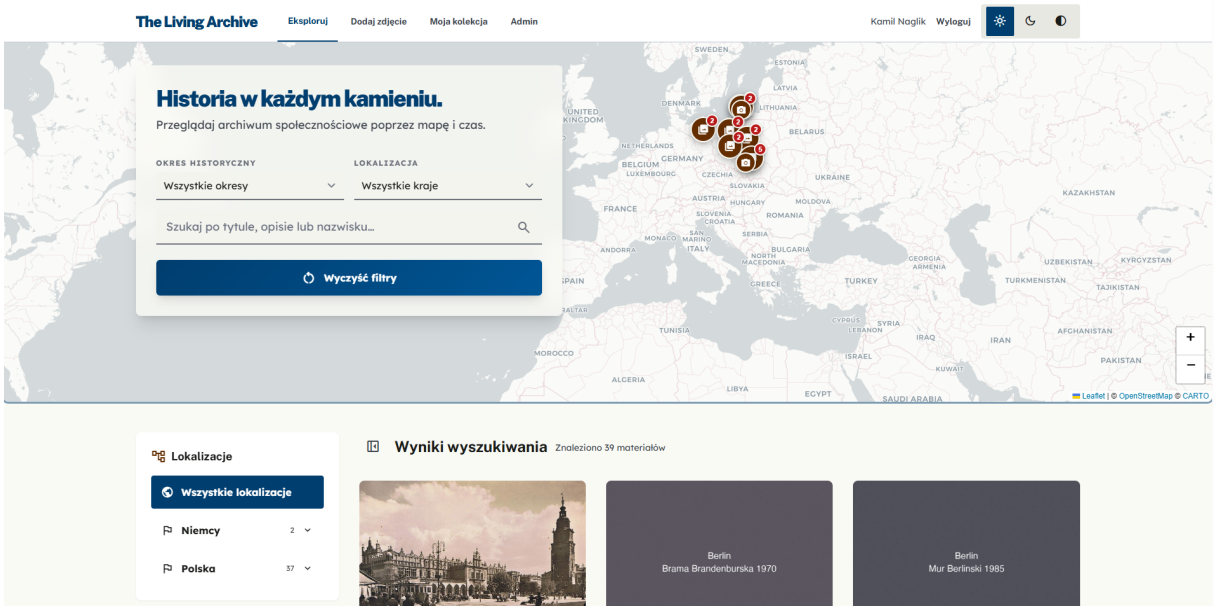
- **Mapa interaktywna** - oznaczona `role="application"` z opisowym `aria-label` instruującym o sposobie interakcji.
- **Wskaźniki ładowania** - `role="status"` z `aria-live="polite"` i ukrytym tekstem `sr-only` dla czytników.
- **Modale** - `role="dialog"`, `aria-modal="true"`, opisowy `aria-label`. Zamykanie przez Escape.
- **Karty zdjęć** - `<article>` z `aria-label`, `tabindex="0"`, obsługa Enter i Space.
- **Drzewo lokalizacji** - `aria-expanded` na rozwijalnych węzłach, `aria-current="location"` na aktywnym.
- **Respektowanie preferencji systemowych** - motyw inicjalizowany na podstawie `prefers-color-scheme` i `prefers-contrast: more`.

11 Podsumowanie

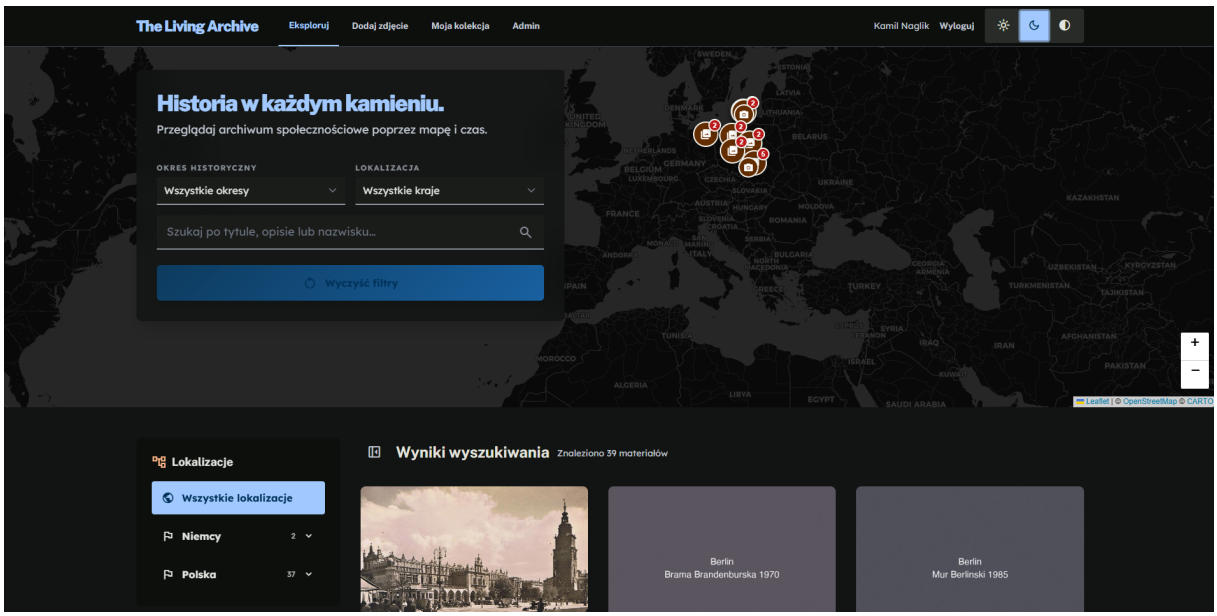
The Living Archive to kompletna aplikacja webowa realizująca cele społecznościowego archiwum cyfrowego. Projekt łączy przemyślaną architekturę informacji (trzypoziomowa hierarchia lokalizacji, oś czasowa) z nowoczesnym stosem technologicznym (Angular 19, .NET 8, PostgreSQL). System zapewnia bezpieczeństwo (OAuth 2.0, JWT, polityki autoryzacji), dostępność (WCAG 2.1 AA) oraz intuicyjny interfejs użytkownika z trzema motywami kolorystycznymi i pełną responsywnością.

Kluczowe decyzje architektoniczne - Clean Architecture z warstwą serwisów, self-referencyjna tabela lokalizacji, grupowanie markerów na mapie, kaskadowe filtrowanie - zostały podyktowane specyfiką domeny archiwistycznej i potrzebami użytkowników końcowych.

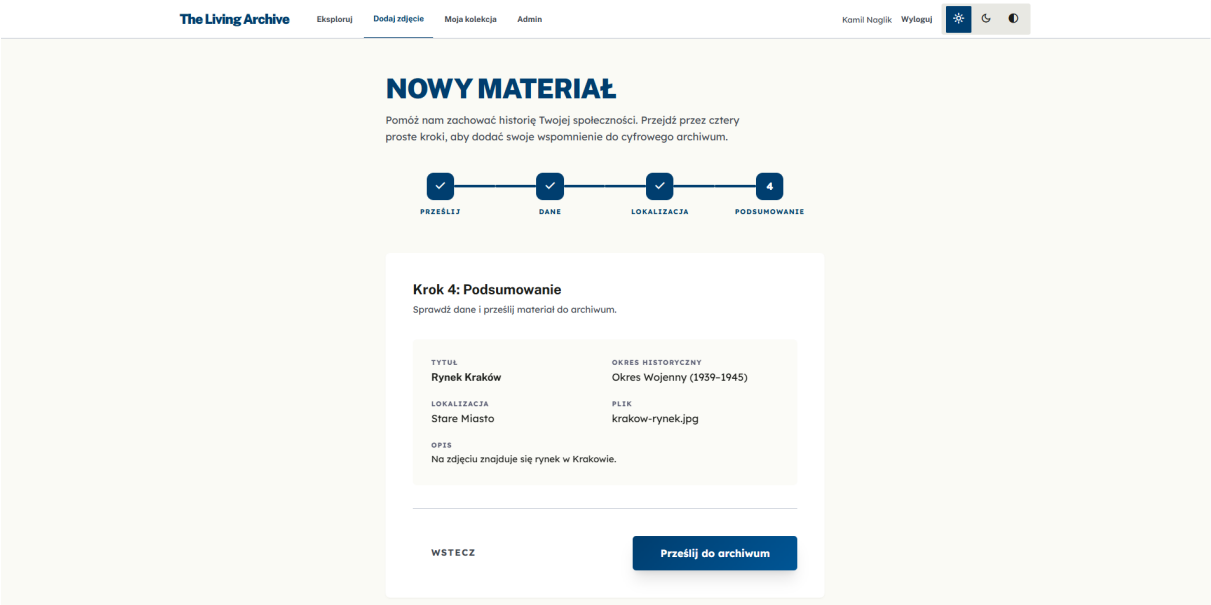
12 Zrzuty ekranu aplikacji



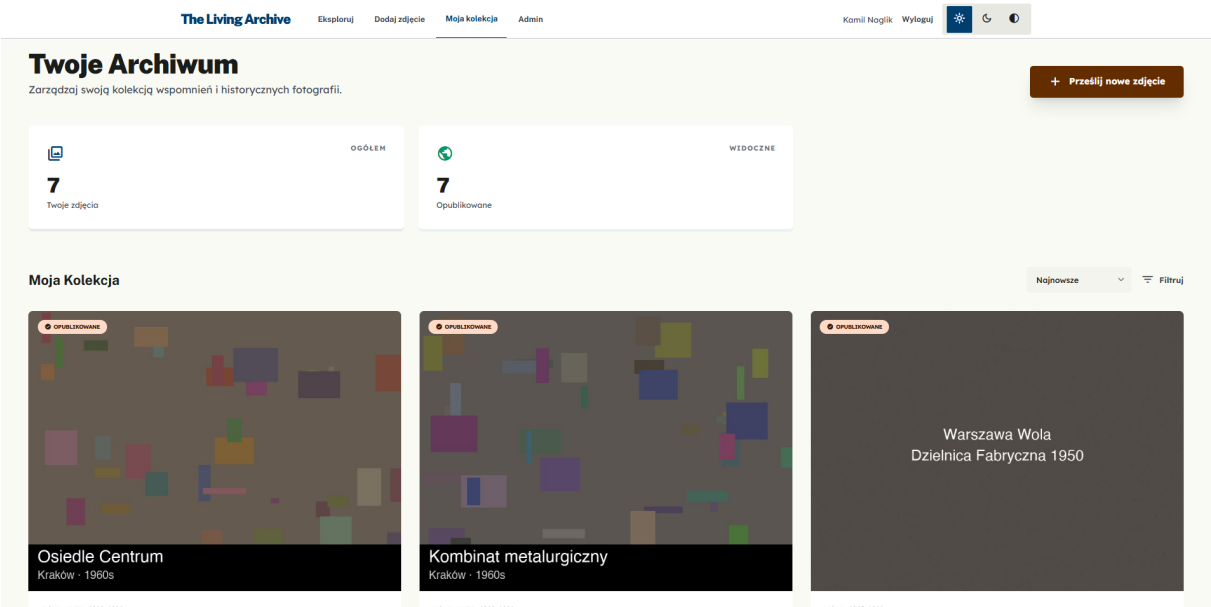
Rysunek 3: Strona eksploracji - motyw jasny



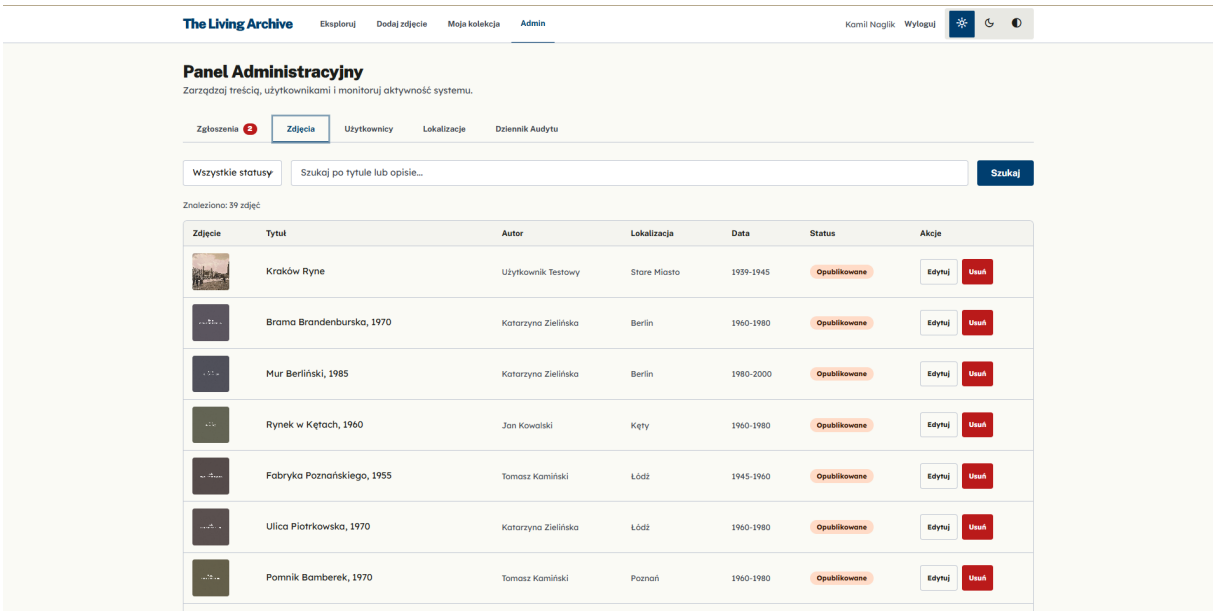
Rysunek 4: Strona eksploracji - motyw ciemny



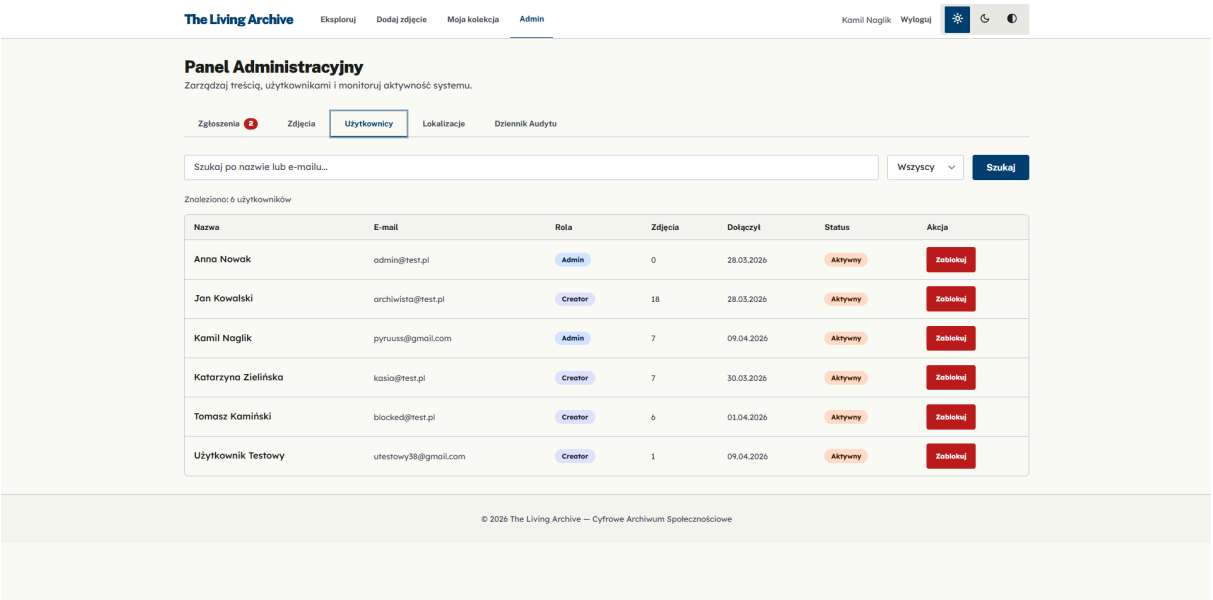
Rysunek 5: Dodawanie zdjęcia - widok podsumowania



Rysunek 6: Kolekcja użytkownika



Rysunek 7: Panel administracyjny - zarządzanie zdjęciami



Rysunek 8: Panel administracyjny - zarządzanie użytkownikami